

This command creates a table called `temp_data` with the same data structure and data as the table called `categories`:

```

forumbd=> select * from temp_categories ;
pk |          title          |          description
----+-----+-----
  1 | Database                | Database related discussions
  2 | Unix                    | Unix and Linux discussions
  3 | Programming Languages   | All about programming languages
  4 | C Language              | Languages
  5 | Python Language        | Languages
  6 | A new discussion        | NULL
(6 rows)

```

Updating data

Now let's try updating some data:

1. If you wanted to change the Unix value to the Linux value, you would need to run the following statement:

```

forumbd=> update temp_categories set title='Linux' where pk = 2;
UPDATE 1

```

This statement will modify the Unix value to the Linux value in the title field for all rows of the `temp_categories` table that have `pk=2`, as seen here:

```

forumbd=> select * from temp_categories where pk=2;
pk | title |          description
----+-----+-----
  2 | Linux | Unix and Linux discussions
(1 row)

```

2. If you wanted to change the title value of all the lines for which the description value is Languages, you would need to run the following statement:

```

forumbd=> update temp_categories set title = 'no title' where
description = 'Languages';
UPDATE 2

```

UPDATE 2 means that only two rows have been modified, as shown here:

```

forumdb=> select * from temp_categories order by description;
pk |          title          |          description
----+-----+-----
  3 | Programming Languages | All about programming languages
  1 | Database              | Database related discussions
  4 | no title              | Languages
  5 | no title              | Languages
  2 | Linux                 | Unix and Linux discussions
  6 | A new discussion      | NULL
(6 rows)

```

You must be careful when using the UPDATE command. If you work in auto-commit mode, there is no chance of turning back after the update is complete. Auto-commit is the default in `psql`.

Deleting data

In this section, we will see how to delete data from a table. The command needed to delete data is `delete`. Let's get started:

1. If we want to delete all records in the `temp_categories` table that have `pk=5`, we have to perform the following command:

```

forumdb=> delete from temp_categories where pk=5;
DELETE 1

```

The preceding statement deletes all the records that have `pk=5`. `DELETE 1` means that one record has been deleted. As you can see here, the row with the value of `pk=5` is no longer present in `temp_categories`:

```

forumdb=> select * from temp_categories where pk=5;
pk | title | description
----+-----+-----
(0 rows)

```

2. Now, if we want to delete all rows that have a `description` value equal to `NULL`, we have to execute this statement:

```

forumdb=> delete from temp_categories where description is null;
DELETE 1

```

The preceding statement used a DELETE command combined with the IS NULL operator.

3. If you want to delete all records from a table, you have to execute the following:

```
forumbd=> delete from temp_categories ;
DELETE 4
```



Be very careful when you use this command – all records present in the table will be deleted!

Now the temp_categories table is empty, as shown here:

```
forumbd=> select * from temp_categories;
pk | title | description
----+-----+-----
(0 rows)
```

4. If we want to reload all the data from the categories table to the temp_categories table, we have to execute this statement:

```
forumbd=> insert into temp_categories select * from categories;
INSERT 0 6
```

The preceding statement takes all values from the categories table and puts them in the temp_categories table, as you can see here:

```
forumbd=> select * from temp_categories order by description;
pk |          title          |          description
----+-----+-----
 3 | Programming Languages | All about programming languages
 1 | Database               | Database related discussions
 4 | C Language             | Languages
 5 | Python Language        | Languages
 2 | Unix                   | Unix and Linux discussions
 6 | A new discussion       | NULL
(6 rows)
```

5. Another way to delete data is by using the TRUNCATE command. When we want to delete all the data from a table without providing a where condition, we can use the TRUNCATE command:

```
forumdb=> truncate table temp_categories ;
TRUNCATE TABLE
```

The TRUNCATE command deletes all data in a table. As you can see here, the temp_categories table is now empty:

```
forumdb=> select * from temp_categories;
 pk | title | description
----+-----+-----
(0 rows)
```

Here is some key information about the TRUNCATE command:

- TRUNCATE deletes all the records in a table similar to the DELETE command.
- In the TRUNCATE command, it is not possible to use where conditions.
- The TRUNCATE command deletes records much faster than the DELETE command.

Summary

This chapter introduced you to the basic SQL/PostgreSQL statements and some basic SQL commands. You learned how to create and delete databases, how to create and delete tables, what types of tables exist, which basic statements to use to insert, modify, and delete data, and the first of many basic queries you can use to query the database.

In the next chapter, you will learn how to write more complex queries that relate to multiple tables in different ways.

Verify your knowledge

- On PostgreSQL 15 and PostgreSQL, is it possible to make DDL as a normal user?
No it's not possible. See the *PostgreSQL and the public schema* section for more details.
- What is the psql command to list all the databases with their sizes?

```
postgres=# \l+
```

See the *Confirming the database size* section for more details.

- If the table is defined as the following:

```
create table mytable (id integer,city_name varchar(60));
```

The question is, does the following query show all records for which the `city_name` field is `null`?

```
select * from mytable where city_name = '';
```

No it doesn't. The correct query is:

```
select * from mytable where city_name is null;
```

See the *NULL values* section for more details.

- Can we create a new database, taking an existing one as a starting point?

Yes, we can. We can use the `TEMPLATE` option.

See the *Making a new database from a modified template* section for more details.

- Is the following query is the best way to delete all records in the table called `mytable`?

```
delete from mytable;
```

No, it isn't. The best way to delete all the records in a table is using the `TRUNCATE` statement.

See the *Deleting data* section for more details.

References

- The `CREATE DATABASE` official documentation: <https://www.PostgreSQL.org/docs/current/sql-createdatabase.html>
- The `CREATE TABLE` official documentation: <https://www.PostgreSQL.org/docs/current/sql-createtable.html>
- The `SELECT` official documentation: <https://www.PostgreSQL.org/docs/current/sql-select.html>
- The `INSERT` official documentation: <https://www.PostgreSQL.org/docs/current/sql-insert.html>
- The `DELETE` official documentation: <https://www.PostgreSQL.org/docs/current/sql-delete.html>
- The `UPDATE` official documentation: <https://www.PostgreSQL.org/docs/current/sql-update.html>
- The `TRUNCATE` official documentation: <https://www.PostgreSQL.org/docs/current/sql-truncate.html>

Learn more on Discord

To join the Discord community for this book – where you can share feedback, ask questions to the author, and learn about new releases – follow the QR code below:

<https://discord.gg/jYWCjF6Tku>



5

Advanced Statements

In the previous chapter, we started taking our first steps with PostgreSQL. In this chapter, we will analyze the SQL language more deeply and write more complex queries. We will talk about SELECT/INSERT/UPDATE again, but this time, we will use the more advanced options surrounding them. We will then cover **joins**, **common table expressions (CTEs)**, and **merge** in depth.

The topics we will talk about will be the following:

- Exploring the SELECT statement
- Using UPSERT and MERGE
- Exploring CTEs

Technical requirements

Before starting, remember to start the Docker container named `chapter_05`, as shown below:

```
$ bash run-pg-docker.sh chapter_05
postgres@learn_postgresql:~$ psql -U forum forumdb
```

Exploring the SELECT statement

As we saw in the previous chapter, we can use the SELECT statement to filter our datasets using the equality condition. In the same way, we can filter records using > or < conditions, such as in the following example:

```
forumdb=> select * from categories where pk > 2;
 pk |          title          |          description
----+-----+-----+-----
```



```

3 | Programming Languages | All about programming languages
(1 row)

```

The preceding query returns all records that have `pk > 2`.

Another condition that we can use with the `SELECT` statement is the `like` condition. Let's take a look at this next.

Using the like clause

Suppose we wanted to find all records that have a `title` field value starting with the string `Prog`.

To do this, we would have to use the `like` condition:

```

forumdb=> \x
Expanded display is on.
forumdb=> select * from categories where title like 'Prog%';
-[ RECORD 1 ]-----
pk          | 3
title       | Programming Languages
description | All about programming languages

```

As shown, the preceding query returns all records that have a `title` beginning with the string `Prog`. In a similar vein, if we wanted to find all records with titles ending with the word `Languages`, we would have to write the following:

```

forumdb=> select * from categories where title like '%Languages';
-[ RECORD 1 ]-----
pk          | 3
title       | Programming Languages
description | All about programming languages

```

The two kinds of searches can also be combined. For example, if we wanted to search all records that contain the partial string `discuss`, we would write the following:

```

forumdb=> \x
Expanded display is off
forumdb=> select * from categories where description like '%discuss%';
 pk | title | description
----+-----+-----

```

```

1 | Database | Database related discussions
2 | Unix    | Unix and Linux discussions
(2 rows)

```

The query given here will return all records whose description contains the string `discuss`.

Now let's try to run the following query and see what happens:

```

forumdb=> select * from categories where title like 'prog%';
(0 rows)

```

As we can see, the search does not return any results. This happens because `like` searches are case-sensitive.

Now let's introduce the `upper (text)` function. The `upper` function, given an input string, returns the same string with all characters in uppercase, as here:

```

forumdb=> select upper('prog');
upper
-----
PROG
(1 row)

```



In PostgreSQL, it is possible to call functions without writing `FROM`. PostgreSQL does not need dummy tables to perform the `SELECT` function. If we were in Oracle DB, the same query would have to be written this way: `select upper('prog') from DUAL;`

Returning to our preceding example, if we wanted to perform a `like` case-insensitive search, we would have to write this statement:

```

forumdb=> select * from categories where upper(description) like
'%DISCUSS%';
pk | title | description
----+-----+-----
1 | Database | Database related discussions
2 | Unix    | Unix and Linux discussions
(2 rows)

```

We have now covered all of the functions that can be performed using the `like` operator.